
Python List Operations – Internal Working

Python lists are **dynamic arrays** (not linked lists). Internally, they are implemented using a **contiguous block of memory (array of pointers)**.

This means:

- Elements are stored in **sequential memory locations**
- Each element is a **reference (pointer) to an object**
- When size increases, Python may **resize the array**

📥 Ways to Add Elements in List

---

1. append()

✓ What it does:

Adds element at the **end**

⚙ Internal working:

- Checks capacity
- If space available → directly place element at end
- If not → resize list, then add

🔁 Workflow:

Check space → place at end → update size

⏱ Time: O(1) amortized

---

2. insert(index, value)

✓ What it does:

Adds element at a specific position

⚙ Internal working:

- Check index
- Shift elements right from that position
- Insert element
- Resize if needed

🔁 Workflow:

Find index → shift right → insert → update size

⏱ Time: O(n)

---

3. extend(iterable)

✓ What it does:

Adds multiple elements at end

⚙ Internal working:

- Iterates over given iterable
- Adds each element one by one at end
- May resize once or multiple times depending on size

🔁 **Workflow:**

Loop items → append each → update size

⏱️ **Time: O(k) (k = number of elements)**

❌ Ways to Remove Elements in List

---

1. remove(value)

✓ **What it does:**

Removes first matching value

⚙ **Internal working:**

- Linear search to find value
- Remove it
- Shift elements left

🔁 **Workflow:**

Search → delete → shift left

⏱️ **Time: O(n)**

---

2. pop(index)

✓ What it does:

Removes element at index (default last)

⚙ Internal working:

- Directly accesses index
- Removes element
- Shifts elements left (if not last element)

🔁 Workflow:

Access index → delete → shift (if needed)

⏱ Time:

- O(1) for last element
- O(n) for middle

---

3. del

✓ What it does:

Deletes element or slice

⚙ Internal working:

- Removes reference
- Shifts elements if needed (for index/slice)

🔁 Workflow:

Locate → delete → shift (if required)

Time:O(n)

---

4. clear()

✓ What it does:

Removes all elements

⚙ Internal working:

- Removes all references
- Frees internal array
- Keeps empty list structure

🔁 Workflow:

Remove all references → reset size to 0

⏱ Time: O(n)

📌 Other Python List Operations (Internal Working)

🔢 1. count(value)

✓ What it does:

Counts occurrences of a value in list

⚙ Internal working:

- Starts from index 0
- Compares each element with target value
- Increases counter when match found
- Returns final count

🔁 Workflow:

Traverse → Compare → Count matches → Return result

⏱ Time: O(n)

🔍 2. index(value)

✓ What it does:

Returns first position of value

⚙ Internal working:

- Linear search from left
- Stops at first match

- Returns index
- If not found → error

🔁 **Workflow:**

Scan → Match → Return index → Stop

⏱️ **Time: O(n)**

🔄 **3. sort()**

✓ **What it does:**

Sorts list in ascending order (default)

⚙ **Internal working:**

- Uses **Timsort algorithm (hybrid of Merge Sort + Insertion Sort)**
- Breaks list into small chunks
- Sorts chunks
- Merges sorted parts efficiently

🔁 **Workflow:**

Divide → Sort small parts → Merge → Final sorted list

⏱️ **Time: O(n log n)**

🔄 4. reverse()

✓ What it does:

Reverses list in place

⚙ Internal working:

- Uses two-pointer approach
- Swaps elements from start and end
- Moves inward until middle

🔁 Workflow:

Start ↔ End swap → Move inward → Complete

⏱ Time: O(n)

📋 5. copy()

✓ What it does:

Creates shallow copy of list

⚙ Internal working:

- Allocates new list memory
- Copies references of elements (not deep copy)
- Both lists share same objects

🔁 Workflow:

Allocate new list → Copy references → Return new list

⏱ Time: O(n)

🧹 6. clear()

✓ What it does:

Removes all elements

⚙ Internal working:

- Deletes all references inside list
- Frees internal array logically
- Keeps empty structure

🔁 Workflow:

Remove references → Reset size → Empty list

⏱ Time: O(n)

🧠 7. len(list)

✓ What it does:

Returns number of elements

⚙ Internal working:

- Python maintains a **size counter internally**
- No traversal needed
- Direct value returned

🔁 Workflow:

Read stored size → Return value

⏱ Time: O(1)

🔗 8. + (concatenation)

✓ What it does:

Combines two lists

⚙ Internal working:

- Creates new list
- Copies elements from both lists into new memory
- Old lists remain unchanged

🔁 Workflow:

Allocate new list → Copy list1 → Copy list2 → Return

⏱ Time: O(n + m)

🔁 9. * (repetition)

✓ What it does:

Repeats list multiple times

⚙ Internal working:

- Creates new list
- Copies same references repeatedly

🔁 Workflow:

Allocate → repeat copy → return

⏱ Time: O(n × k)